

Visualizing Price Trends in China: A Multi-Channel Grayscale CNN Approach

Jing Chen

Central University of Finance and Economics

Yushan Xue

Central University of Finance and Economics

Wei Zhao

Central University of Finance and Economics

Yifeng Zhu

Central University of Finance and Economics*

July 2025

*Chen, chenjingma@cufe.edu.cn; Xue, cnxueyushan@163.com; Zhao, 2023210996@email.cufe.edu.cn; Zhu (corresponding author), suxian2000@gmail.com. We thank Dacheng Xiu and Guofu Zhou for helpful comments. Chen acknowledges the financial support from the National Natural Science Foundation of China (No. 12001556) and the Program for Innovation Research in Central University of Finance and Economics. Chen and Xue appreciate the financial support from the National Key Research and Development Program of China under the “National Key R&D Program of China” (No. 2023YFF0614700), Zhu gratefully acknowledges the financial support from the Humanities and Social Science Fund of Ministry of Education of China (Grant No. 24YJA790106), the Fundamental Research Funds for the Central Universities, and the Key Project of Philosophy and Social Sciences Research, Ministry of Education, China (No. 22JZD011). All errors remain our responsibility.

Visualizing Price Trends in China: A Multi-Channel Grayscale CNN Approach

Abstract

This study examines data from the A-share market, utilizing a convolutional neural network (CNN) with two-channel grayscale images to predict stock price trends. The findings indicate that increasing the information content and the number of channels enhances the high-minus-low (H-L) return spread and the Sharpe ratio (SR) of stock deciles. Results show that the model significantly improves performance, particularly regarding H-L weighted returns and SR. Additionally, we confirm the positive impact of incorporating more information and channels on the CNN model performance. This work provides new insights into applying deep learning in stock market prediction and serves as a valuable reference for future research in the field.

JEL Classification: G11

Keywords: Convolutional neural network, Multi-channel grayscale image, A-shares

1 Introduction

Stock price prediction, a core research area in finance, has long attracted attention from scholars and investors. With the ongoing development of financial markets, accurately forecasting the future movement of stock prices has become essential for investors and critical for risk management, asset allocation, and various other aspects of financial institutions. In recent years, driven by rapid technological advancements and the increasing availability of data, models and methods for predicting stock price fluctuations, significant evolution and improvement have been observed.

Early stock price prediction primarily relied on two traditional methods: fundamental analysis and technical analysis. The founder of value investing, Graham (1934), outlined the principles of value investing in his seminal work, *Securities Analysis*, emphasizing that fundamental factors determine the value of stocks. He also provided detailed methods for evaluating companies based on financial data and market conditions, and over time, fundamental analysis has evolved. With advancements in financial statement analysis, investors are better equipped to assess a company's financial health and profitability. Additionally, macroeconomic factors' influence on financial markets has gained increasing recognition. Investors have begun focusing on indicators like economic growth, inflation, interest rates, and how industry trends and competitive dynamics affect corporate value. Fundamental analysis is based on essential information like financial statements and macroeconomic indicators, where investors assess a company's intrinsic value to determine the reasonableness of stock prices; however, fundamental analysis often involves vast amounts of data and may be slow to reflect market changes, making it more suitable for long-term investment strategies. Moreover, investors may interpret the same information differently, influenced by their experiences, cognition, and values.

The Dow Theory influences technical analysis based on market behavior data like historical price trends and trading volumes. It also includes derived indicators like moving averages (MA), relative strength index (RSI), and stochastic oscillators (KDJ) to forecast future price movements. Technical analysis uses charting techniques, such as line and bar charts, to help investors visually track price trends and patterns, potentially identifying signals for future market movements;

however, these patterns often struggle to capture modern markets’ complex, nonlinear dynamics.

Markowitz (1952) introduced the mean-variance model to determine the optimal investment portfolio, marking the establishment of the modern portfolio theory. Sharpe (1964) built upon this framework by developing the capital asset pricing model (CAPM) based on the mean-variance approach. Fama and French (1993) expanded on the CAPM by introducing the three-factor model, which has since been refined and evolved into a multi-factor model. These models share common factors like size, momentum, volatility, liquidity, and technological factors; however, this factor proliferation presents two key challenges for traditional methods: factor selection and modeling their linear or nonlinear relationships (Li et al., 2019).

Selecting factors in multi-factor models directly influences the model’s explanatory power and predictive accuracy; thus, factor selection must consider the economic rationale behind each factor and the balance between explanatory power and simplicity to avoid redundancy from highly correlated factors. For example, in the Fama–French three-factor model (Fama and French, 1993), the market, size, and value factors each have clear economic interpretations. In their five-factor model, Fama and French (2015) addressed the correlations between new and existing factors to minimize redundancy. The advent of machine learning and deep learning has fostered new approaches to address the challenge of modeling complex linear and nonlinear relationships between factors (Borges et al., 1977; Fischer and Krauss, 2018).

The convolutional neural network (CNN) model is based on multi-channel grayscale images and offers a novel approach to stock price prediction. Borges et al. (1977) pointed out that human learning is often more effective when dealing with visual information; thus, image-based methods may be better suited to capturing complex patterns than traditional time series methods. CNNs are widely used in image recognition and understanding and are particularly effective at learning hidden information and patterns from input images.

Compared to single-channel data, multi-channel images offer a richer, more comprehensive representation of data. This study’s multi-channel grayscale images consist of the opening price, highest price, lowest price, closing price, trading volume, etc. Integrating information from multiple dimensions enables these images to discover correlations among stock factors. Unlike the raw data, the multi-channel grayscale images in this paper are scaled within a window before input into the model. This approach allows the model to focus on the relative relationships of

each variable within the 20-day time window rather than the absolute values. The key to utilizing multi-channel grayscale images effectively is how the stock data is represented in multiple dimensions. As explored in this study, stock price prediction represents another application of convolutional neural networks in the financial domain.

Due to financial markets' complexity and nonlinear nature, traditional statistical methods often struggle to identify the underlying patterns. Conversely, deep learning models — particularly convolutional neural networks (CNN), recurrent neural networks (RNN), and long short-term memory networks (LSTM) — have emerged as powerful tools for financial data analysis, given their robust feature learning capabilities (Zhao and Xue, 2021; Zhu, 2020; Xie, 2019). Domestic and international research indicates that deep learning has been widely applied in areas like stock market prediction, risk management, and asset allocation (Jiang, 2021; Chen et al., 2021; Zhao et al., 2020). These models can automatically extract relevant features from large volumes of historical data, eliminate the need for manually defined rules, and demonstrate strong predictive power.

CNNs, for example, are widely used in image classification and object recognition and find applications in speech recognition and natural language processing (Li et al., 2016). The success of convolutional neural networks can be attributed to several key factors:

1. **Automatic extraction of local features:** Convolutional layers automatically extract local features from images, eliminating the need for complex manual feature design; thus, the model can handle large-scale, high-dimensional data more effectively. Using pooling layers further reduces feature dimensionality, helping avoid the complexities associated with traditional feature extraction methods.
2. **Weight sharing mechanism:** Convolutional layers employ a weight-sharing mechanism, where multiple neurons share the same convolutional kernel parameters. This approach significantly reduces the number of parameters, enhancing the model's computational efficiency.
3. **Utilization of spatial structure:** CNNs effectively exploit the spatial structure of the input data. In images, pixels have inherent spatial relationships; the sliding operation of the convolutional kernels preserves these local spatial correlations, allowing the model to capture more meaningful patterns.

4. **Capturing nonlinear relationships:** Convolutional neural networks can apply nonlinear activation functions (such as ReLU, leaky ReLU, or Sigmoid) after the convolutional layers to capture complex, nonlinear relationships in data. These activation functions introduce nonlinearity, enabling the model to learn more intricate patterns.

This study explores how to convert financial market data (such as stock prices and trading volumes) into multi-channel grayscale images and leverage CNNs for classifying and predicting stock price trends. The research encompasses several key aspects: collecting and preprocessing stock data, generating multi-channel grayscale images, designing and optimizing CNN models, model training and prediction, and evaluating and comparing model performance. Based on 30 repeated trials, the experimental results demonstrate that the model can significantly improve performance under various weighting strategies, particularly regarding high-minus-low (H-L) weighted returns and the Sharpe ratio. Furthermore, this study compares single-channel images with additional information against models with increased channel numbers in the same 30 repeated experiments. This approach highlights the beneficial impact of adding information and increasing the number of channels, offering valuable insights for future enhancements.

Despite continuous advancements in technology, stock market forecasting remains a complex challenge. First, Malkiel (2003) argued that the market is efficient and that stock prices already incorporate all available information, making it extremely difficult to predict short-term price movements accurately. Conversely, Grossman and Stiglitz (1980) discussed the issue of information asymmetry, where institutional investors typically possess more comprehensive information than individual investors; thus, they can exert significant market influence through large-scale buying and selling, creating an environment where markets cannot be entirely efficient, and predicting prices with high accuracy remains elusive. Moreover, macroeconomic factors, political events, and company-specific news, which are often unpredictable, often influence stock prices. The presence of “noise” in market data further complicates capturing price fluctuations, adding a layer of uncertainty to model construction.

Second, financial time series exhibit nonlinearity and complex dependencies that simple linear models struggle to capture (Zhang, 2003). While historical data influence stock prices, the relationships between variables are often intricate and involve long-term interactions that simple models fail to represent effectively.

Lastly, the issue of model overfitting has become increasingly prominent with the growing use of machine learning and deep learning techniques in stock prediction. Gu, Kelly, and Xiu (2020) pointed out that these models may achieve high accuracy on historical data; however, their predictive performance often deteriorates in real-market conditions. Overfitting reduces model robustness, especially when confronted with unseen data, significantly reducing performance.

The advantages of CNNs have attracted attention in stock market prediction, and their integration with financial data has become a key trend in recent years. Ding et al. (2015) applied deep CNNs to model the short- and long-term effects of events on stock price movements in their event-driven stock market prediction research. Converting financial data into image structures and using CNNs for classification or regression tasks has also emerged as a promising approach. For example, Gudelek et al. (2017) employed a sliding window technique to capture trend and momentum indicator images for 17 ETFs, predicting the following day’s price fluctuations with an accuracy of 72%. Hu et al. (2018a) utilized candlestick charts and CNNs to cluster stocks. Similarly, Jiang, Kelly, and Xiu (2023) used data from the US stock market between 1993 and 2000 to generate OHLCV (open, high, low, close, and volume) grayscale images with a 20-day window, incorporating the opening price, highest price, lowest price, closing price, 20-day moving average, and trading volume. They then trained a CNN model to classify whether prices would rise or fall over the next 20 days. Compared to Hu et al. (2018a)’s candlestick chart, the OHLCV format conveys the same information with fewer channels and smaller images. Based on the OHLCV and CNN model, Jiang, Kelly, and Xiu (2023) achieved an accuracy of 53.3% on the test set from the US stock market between 2001 and 2019, demonstrating that using image-based representations and CNNs to predict future price fluctuations is viable. Multi-channel imaging has become a relatively novel and promising research direction in stock market prediction.

Based on the relevant literature, using convolutional neural networks (CNNs) with multi-channel grayscale images to predict stock price movements is a promising approach. This method preserves the relative information of the time series data and offers a new analytical perspective; however, several challenges remain. First, appropriately selecting and constructing the images is crucial in converting historical stock data into multi-channel grayscale images. Stock data typically includes multiple dimensions, such as opening price, closing price, highest price, lowest price, and trading volume. When mapping these dimensions to images, it is essential to represent

these signals. Additionally, minimizing noise within the images is critical to avoid obscuring important patterns. Different factors can be expressed in various ways when constructing images, so it is important to design the images to ensure that the information from each dimension remains distinct and interpretable. Finally, deep learning models, including CNNs, are often called “black box” models. While they can deliver high-precision predictions, the reasoning behind these predictions is not easily understood. Investors need to rely on the model’s forecast in stock market prediction and understand the factors driving these predictions. This lack of interpretability poses a significant challenge, as transparency is crucial for trust and decision-making in financial contexts.

The remainder of this article is structured as follows. Section 1 describes constructing multi-channel grayscale images based on the A-share market. Section 2 covers convolutional neural networks’ theoretical foundation and model architecture, while Section 3 presents the model’s training details and empirical results. Section 4 compares the results of adding more information versus increasing the number of channels, as well as the performance of the 1D-CNN. Section 5 introduces the sensitivity analysis and the Monte Carlo simulations in Section 6. Finally, Section 7 concludes the paper.

2 Construction of multi-channel grayscale images based on the A-share market

2.1 Data

This study uses daily stock data from the Wind Economic Database, covering 5,457 stocks listed on the Shenzhen Stock Exchange, Shanghai Stock Exchange, and Beijing Stock Exchange from January 4, 2000, to January 5, 2024. The dataset includes daily indicators such as the opening price, highest price, lowest price, closing price, trading volume, and other relevant metrics. The implementation of standardized financial reporting regulations in China did not take full effect until around 1999 (Liu et al., 2019); therefore, this study selects data from after 2000. All price data used in this study are adjusted for these corporate actions to eliminate the impact of events such as dividends, rights issues, or stock splits on stock prices and facilitate long-term analysis

and comparison. In addition to the raw data provided by Wind, this study applies the following formula:

$$Ret_t = \frac{Close_t - Close_{t-20}}{Close_{t-20}} \times 100\%. \quad (1)$$

To calculate the 20-day return, $Close_t$ and $Close_{t-20}$ are the closing prices for time t and $t-20$, respectively.

2.2 Preprocessing

This study must create a two-channel grayscale image using the raw data provided by Wind, which includes the opening price, highest price, lowest price, closing price, trading volume, and the 20-day return.

We plotted the stock data every 20 days to convert stock market data into images suitable for input into a CNN model; however, the generated images have different starting points due to the varying start dates of stock trading for different stocks. This misalignment of the date windows can cause deviations when calculating cumulative returns. For example, suppose the start dates of some stocks are not synchronized in each adjustment cycle. In this case, the cumulative return may be calculated multiple times, leading to an inflated cumulative return and inaccurate evaluation results. Therefore, we must align the date windows of all stock data to ensure that the starting point for every 20-day adjustment is consistent across all stocks. This study performed a date window alignment operation during the data preprocessing stage to address the above mentioned issues. The starting date for the images was set to specific dates (e.g., January 4, 2000; February 15, 2000; March 14, 2000, etc.). This approach ensures that each image starts from one of these predefined points, generating an image every 20 days. For stock data with deviations in start dates, missing dates were either truncated or filled to ensure consistency with data from other stocks, creating a unified time window. This alignment process ensured that the performance of all stocks within the same adjustment cycle was correctly calculated, avoiding yield discrepancies caused by misaligned time windows and ensuring the accuracy of the cumulative returns. Furthermore, this study rigorously cleaned the raw data during preprocessing to ensure data quality and analysis accuracy to account for the stock market’s shell effect, liquidity issues, and initial public offering

(IPO) effects. This study’s approach is based on the methodology of Liu et al. (2019) and includes the following steps:

1. **Exclusion of stocks with the smallest market capitalization:** Stocks in the lowest 30% of market capitalization at the start of each image are excluded to mitigate the impact of the shell effect. This step helps reduce the noise caused by the significant price fluctuations of small-cap stocks.
2. **Exclusion of stocks with insufficient listing time:** Stocks that have been listed for less than six months are excluded to avoid IPO effect interference. Newly listed stocks typically experience high volatility, and abnormal factors often influence their prices in the early stages of trading.
3. **Exclusion of low-liquidity stocks:** In each 20-day image, stocks with fewer than or equal to 10 trading days are excluded, ensuring that only stocks with sufficient liquidity remain and improving the stability of the analysis and the reliability of the conclusions.

Through these data-cleaning steps, this study minimizes the impact of abnormal market data, ensuring that the analysis accurately reflects real market conditions.

This study selects the research variables for the daily opening price, highest price, lowest price, closing price, trading volume, 20-day moving average, and 20-day return rate of the Chinese A-share market. The daily price data — opening, highest, lowest, and closing price — provide fundamental information about daily stock trading and reflect stock volatility (Tsay, 2005). The opening price represents investors’ initial expectations for the market, while the highest and lowest prices indicate the extent of price fluctuations throughout the day. However, the closing price reflects the final market valuation of the stock at the end of the trading day. These price data offer a comprehensive view of intraday price behavior, serving as a crucial foundation for subsequent trend analysis.

As a direct indicator of market activity, trading volume provides insights into investors’ buying and selling intentions and market liquidity levels. Many studies have examined the relationship between trading volume, price fluctuations, and trend reversals. For example, Karpoff (1987) reviewed the significance of trading volume in forecasting and found that it positively correlated with both the magnitude of price changes and the direction of price movements. Therefore,

incorporating trading volume into the analysis can help to capture shifts in investor behavior and liquidity risks in the market.

The 20-day moving average is a widely used indicator in technical analysis and is a key reference for short-term market trends. By averaging past price data, the moving average smooths out price fluctuations and reduces noise, making it a better reflection of market continuity and potential reversals. This characteristic of “remembering” past trends makes it a valuable tool for trend analysis. The 20-day moving average also incorporates more historical price information, enhancing the model’s ability to predict future price movements.

The 20-day yield measures stock performance over a specific period and can help reveal historical return trends. This variable reflects the recent profitability of a stock and provides a basis for predicting future returns. By incorporating the 20-day return rate, we gain a more comprehensive understanding of a stock’s return characteristics and can capture the dynamic changes in short-term performance.

These variables comprehensively describe the stock market by capturing key aspects such as price, trading volume, trends, and returns. This multidimensional approach enhances the model’s ability to accurately characterize and predict market trends.

2.3 Construction of multi-channel grayscale images and corresponding labels

Plotting all the above variables on the same graph or within a single channel would result in overlapping information, leading to high information density and increased interference between features, particularly between yield and price data. This mixture of information blurs the boundaries between different variables and complicates the model’s learning process, negatively affecting feature recognition. To address this issue, this paper proposes using multi-channel grayscale images. By mapping variables to separate channels, we can avoid information overlap, enhancing the model’s ability to recognize distinct features and improving its learning efficiency. This study divides the variables into several channels by calculating the correlation coefficient matrix between the variables using Jensen et al. (2023)’s feature classification method. Based on this matrix, hierarchical clustering is performed using the Ward clustering method. The core idea of Ward clustering is to merge clusters to minimize the increase in the sum of squares within each cluster.

The sum of the squares within each cluster is calculated as follows:

$$SSE(C_i) = \sum_{x \in C_i} \|x - \mu_i\|^2. \quad (2)$$

Here, C_i is the cluster, and μ_i is the centroid of C_i . Ward's method minimizes the increase in the sum of squares, resulting in dense, uniform clusters that are robust to outliers. Based on the clustering results, the variables were grouped into two categories shown in Figure 1: daily opening price, highest price, lowest price, closing price, and 20-day moving average as one group, and trading volume and 20-day return as another. The order of the channels was not considered in the image generation process.

[Insert Figure 1 about here]

When constructing the images, this study follows the approach of Jiang, Kelly, and Xiu (2023), setting the height of each channel to 64 pixels and using a width of 3 pixels to represent daily data, resulting in a total image width of 60 pixels. The image background is black (represented by 0), and the data are drawn in white (represented by 255). In the first channel, OHLC bars represent the daily opening, highest, lowest, and closing prices, with the 20-day moving average plotted at the middle pixels of each day, connecting these points into a line. Specifically, the data within the 20-day window is scaled such that the opening price, highest price, lowest price, and the maximum value of the 20-day moving average are positioned at the top of the image, while the minimum value is at the bottom.

In the second channel, the yield curve is drawn in approximately the top four-fifths of the image, using the same method as for the 20-day moving average. The daily trading volume is represented as a bar chart in the bottom four-fifths of the image, with the maximum trading volume scaled to the top of this region. A 1-pixel black line separates the yield curve and the trading volume section to separate the two components. If there are missing values within the window, they are left blank in the corresponding positions. Figure 2 shows the resulting multi-channel grayscale image.

[Insert Figure 2 about here]

This study uses the price movement (rise or fall) on the 20th day after the corresponding date

as the label for each image.

3 Theoretical basis of CNNs

CNNs are deep learning models inspired by the visual system in biology; they are designed to simulate how the human brain processes visual information. The visual cortex processes visual stimuli and contains neurons that respond strongly to specific optical features, such as edges, angles, and motion. This hierarchical information processing in the brain inspires the architecture of CNNs. The multilayer convolution operations in a CNN simulate the hierarchical processing of visual information by extracting features from the images. Each convolutional kernel in a CNN functions like a neuron in the visual cortex, responding to local regions of the image, known as local receptive fields. Just as neurons in the visual system respond to the same features (e.g., edges or lines) across the entire field of view, each convolutional kernel applies the same operation at different positions in the image. In the early layers, CNNs detect simple features like edges or colors. As the network deepens, it combines these simple features to recognize more complex shapes and objects, similar to how the biological visual system recognizes and processes visual information through hierarchical stages.

This biological inspiration gives CNNs powerful capabilities in processing visual data, leading to their widespread application in fields such as image classification, object detection, and image segmentation. This section describes each component of the CNN architecture used in our study.

3.1 Basic structure of Convolutional Neural Networks

CNNs generally comprise an input layer, convolutional layers, pooling layers, activation functions, fully connected layers, and an output layer. The input layer receives raw data, while the convolutional layers (the core of the CNN) extract local features by scanning the input data with convolutional kernels to compute feature maps. The pooling layers perform downsampling on these feature maps, reducing the data dimensionality and computational complexity while preserving the essential features. Activation functions introduce nonlinearity, allowing the model to learn complex patterns in the data. The fully connected layer at the network's end maps the high-level features to the final classification or regression task. In this layer, each neuron is connected

to all neurons in the preceding layer, enabling the model to capture global information. Finally, the output layer generates the model's prediction. These layers work together to extract features from the input data and produce the model's final predictions.

3.1.1 Input layer

The input layer receives raw data, such as images, audio, or other data formats. In image processing, the input layer typically receives a pixel matrix of height $\times$ width dimensions for a single-channel grayscale image, with pixel values ranging from 0 to 255. The input for a three-channel color image is a three-dimensional tensor, typically height $\times$ width $\times$ 3, representing color channels (such as RGB). This input data is then passed to the subsequent layers of the network for further processing.

3.1.2 Convolutional layer

The convolutional layer is the core of the CNN. Its main function is to perform convolution operations on the input feature map, extracting spatial features through convolutional kernels. For example, suppose a three-dimensional tensor I of size $H \times W \times C$ is input, and the convolution kernel K has dimensions $k_h \times k_w \times C$. In this case, the convolution kernel slides across the feature map and performs a matrix inner product with the corresponding section of the feature map. The output of the convolutional layer can be expressed as follows:

$$O(i, j) = \sum_{c=1}^C \sum_{m=1}^{k_h} \sum_{n=1}^{k_w} I(i + m - 1, j + n - 1, c) \cdot K(m, n, c) + b, \quad (3)$$

where $O(i, j)$ is the output feature map value at position (i, j) ; $I(i + m - 1, j + n - 1, c)$ is the pixel value at position $(i + m - 1, j + n - 1)$ in the c -th channel of the input tensor; $K(m, n, c)$ is the weight matrix of the convolution kernel at position (m, n) in the c -th channel; and b is the bias term.

The number of channels in the convolution kernel must match the channels in the input feature map. In the example above, if only one convolution kernel exists, the resulting feature map will have only one channel. Different convolution kernels can capture different types of features in the input data. Therefore, using multiple convolution kernels allows more diverse features to be

extracted, which enhances the model’s feature extraction ability. For example, if D convolution kernels are applied to a three-dimensional tensor I , then D feature maps are obtained.

The width and height of the output feature map depend on the width and height of the input feature map and the convolution kernel, as well as on factors such as the stride of the convolution kernel, the dilation coefficient, and the padding.

1. **Stride:** The stride controls how much the convolution kernel moves across the input feature map. For example, in PyTorch’s Conv2d, setting $\text{stride}=(3,1)$ means the convolution kernel moves down by 3 pixels vertically and to the right by 1 pixel horizontally.
2. **Dilation:** The dilation coefficient controls the spacing between the elements in the convolution kernel. For instance, setting $\text{dilation}=(2,1)$ means that the kernel elements have a one-pixel interval between them in the vertical direction, and in the horizontal direction, the elements are adjacent. Dilated convolution increases the receptive field without increasing the kernel size, thereby capturing a larger range of information.
3. **Padding:** Padding determines whether extra pixels are added around the input to control the size of the output feature map. For example, setting $\text{padding}=(12,1)$ adds 12 rows of pixels vertically and 1 column of pixels horizontally. The default value for padding is 0, meaning no additional pixels are added.

The formula for calculating the output feature map’s width and height by integrating the input feature map’s width and height, the width and height of the convolution kernel, the stride, the dilation coefficient, and the padding can be expressed as:

$$\text{output size} = \left\lfloor \frac{\text{input size} + 2 \times \text{padding} - \text{dilation} \times (\text{kernel size} - 1) - 1}{\text{stride}} + 1 \right\rfloor. \quad (4)$$

3.1.3 Activation function

The activation function introduces nonlinearity into the network, allowing it to learn complex nonlinear relationships between the input and output. Without an activation function, the model would behave like a linear regressor, regardless of the network’s depth, limiting its ability to capture complex patterns.

In practice, the feature maps output by the convolutional layers typically require activation functions for further processing. One of the most commonly used activation functions is the rectified linear unit (ReLU), defined as follows:

$$f(x) = \max(0, x). \quad (5)$$

ReLU replaces all negative values in the feature map with zero while keeping the positive values unchanged. This approach introduces nonlinearity while being computationally efficient. ReLU has become popular in deep learning due to its simplicity, speed, and ability to mitigate the vanishing gradient problem often encountered in deep networks.

Although the ReLU function is piecewise linear, it introduces nonlinearity into the network by including an inflection point at $x = 0$. ReLU only needs to check whether the input is greater than 0; thus, it can be computed quickly. In the region where $x > 0$, the gradient of ReLU is 1, which helps avoid the vanishing gradient problem encountered in activation functions like Sigmoid and Tanh.

Nonetheless, ReLU can suffer from a problem known as neuron death. If many neurons have negative inputs, their outputs will remain constant at 0, preventing them from participating in training and leading to dead neurons. This issue can hinder the model’s learning process. Leaky ReLU introduces a slight slope in the negative region to address this, ensuring that neurons with negative inputs can still contribute to training. The formula for leaky ReLU is:

$$f(x) = \begin{cases} x, & \text{if } x \geq 0 \\ \alpha x, & \text{if } x < 0 \end{cases}. \quad (6)$$

Here, α is a small positive constant, typically set to 0.01. Leaky ReLU retains the benefits of ReLU, such as computational efficiency and avoiding the vanishing gradient problem. At the same time, it mitigates the risk of neuron death.

3.1.4 Pooling layer

The convolutional layers produce large-scale feature maps, and processing them can be computationally expensive. Additionally, adjacent pixels in the feature map often contain redundant

information. To address this, pooling layers are used to downsample the feature maps, reducing their dimensionality and computational complexity while preserving the important features. Pooling helps minimize the risk of overfitting by focusing on the most significant information in the feature maps.

Max pooling and average pooling are common pooling operations. In both cases, the pooling window slides over each channel of the feature map. The maximum value in the region covered by the pooling window is selected for max pooling, while for average pooling, the average value of the region is selected. Pooling is performed independently for each channel of the feature map, meaning that the number of channels remains unchanged.

The input size, the kernel size, and the stride of the pooling window determine the output size of the pooling layer. If no padding is applied, the output size can be calculated as follows:

$$output\ size = \left\lfloor \frac{input\ size - kernel\ size}{stride} \right\rfloor + 1, \quad (7)$$

where *Kernel size* refers to the width or height of the pooling window; and *Stride* is the step size by which the pooling window moves.

3.1.5 Fully connected layer

The fully connected (FC) layer is located at the network's end and maps high-level features from the preceding layers to the final classification or regression task. In the fully connected layer, each neuron is connected to all neurons in the previous layer, and each connection has a learnable weight and bias term. The output is computed through a linear transformation, which the following formula can represent:

$$\mathbf{y} = W\mathbf{x} + \mathbf{b}, \quad (8)$$

where $\mathbf{y}$ is the output vector of the fully connected layer; W is the weight matrix; $\mathbf{x}$ is the input vector (which is typically the flattened output from the convolutional layers); and $\mathbf{b}$ is the bias vector.

Dropout is commonly used in the fully connected layer to prevent overfitting and improve the model's generalization ability. Dropout works by randomly setting the fraction of neurons to zero during each training step. This approach uses a probability p , meaning that some neurons are

“dropped out” and do not participate in the current forward or backward propagation, and their weights are not updated for that training session. By doing this, the network becomes a slightly different subnetwork for each training step, helping to prevent the network from becoming too reliant on any particular neuron or set of neurons. At the test time, dropout is not applied. All neurons are used during the forward pass, allowing the model to perform normally, utilizing all learned parameters.

3.1.6 Output layer

The output layer is the final layer of a neural network; its primary function is to convert the results of the previous layers (usually the fully connected layer) into the final output. In a classification task, the output layer typically produces a probability distribution over the possible classes, often using a softmax activation function for multiclass classification. A sigmoid activation function is commonly used for binary classification to output a probability value. However, the output layer for a regression task typically produces one or more continuous values without any activation function or with a linear activation, depending on the nature of the problem.

The softmax function is a commonly used activation function in classification tasks that converts the network’s output into a probability distribution. It transforms multiple unnormalized real values (logits) into probabilities. Each probability falls between $[0, 1]$, and the sum of all probabilities equals 1. The mathematical expression for the softmax function is:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}. \quad (9)$$

where z_i represents the output for class i , and the denominator is the sum of the exponentials of all output values. Softmax allows the model to determine the most likely class for the input by comparing the probabilities across all categories. The class with the highest probability is typically selected as the final classification result.

3.2 Basic structure of SE module

In CNNs, all input feature map channels are typically processed equally; however, in practice, the contribution of different channels to the final task may vary. Introducing an interchannel

attention mechanism can enhance the network’s ability to evaluate and emphasize the importance of various features. One such mechanism is the squeeze and excitation (SE) module, a lightweight attention mechanism proposed by Hu et al. (2018b). The core idea behind the SE module is to learn and assign weights to each channel based on its importance. SE allows the model to focus more on the channels with the most relevant features for the task, thereby improving its overall performance.

The SE module comprises two stages: squeeze and excitation. Let the input feature map be $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$.

1. **Squeeze:** The spatial dimension of each channel is compressed into a scalar using global average pooling. For each channel c , the pooled output is calculated as follows:

$$z_c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W X_{c,i,j}. \quad (10)$$

The resulting vector z represents the global spatial information across all channels.

2. **Excitation:** The vector z undergoes a fully connected compression, reducing the dimensionality to alleviate the computational complexity. Negative values are suppressed using the ReLU activation function. A fully connected restoration is then performed to recover the dimensionality, followed by applying the Sigmoid function to generate the channel weights s , where each weight lies between $[0, 1]$. The mathematical expression of this process is:

$$\mathbf{s} = \sigma(W_2 \cdot \text{ReLU}(W_1 \cdot \mathbf{z})). \quad (11)$$

$\mathbf{s}$ is the vector of the channel weights. Finally, the original feature maps are scaled by the corresponding channel weights:

$$\tilde{X}_c = X_c \cdot s_c. \quad (12)$$

This process allows the model to focus on the more relevant channels, enhancing its feature extraction ability.

Introducing attention mechanisms between channels enables the network to automatically assign different weights to each channel based on the importance of the input data. This process

highlights useful features while suppressing irrelevant or unimportant ones. The dynamic weighting of channels helps the network reduce redundant information, retain only the most pertinent features, and mitigate the risk of overfitting, ultimately improving model performance.

3.3 Model construction

A CNN model was developed using multi-channel grayscale images to process stock information. This novel CNN-based model is specifically designed to handle dual-channel grayscale image data. The model extracts features independently for each channel and jointly between channels, incorporating fusion operations to capture the mutual information within and between the channels.

This structure enables comprehensive feature extraction through shared convolution operations while preserving the independence of each channel’s information. The model receives dual-channel input data and utilizes separate convolution kernels for each channel, performing convolution operations independently to extract features from each channel. Additionally, joint convolution operations capture mutual information between the two channels.

The features extracted from both channels are then concatenated to form a comprehensive set of features, which fully connected layers process to generate the final target prediction. Figure 3 shows a schematic diagram of the model structure.

[Insert Figure 3 about here]

- **Input Layer:** The model accepts dual-channel grayscale images with a shape of (64, 60, 2). The first dimension (64) represents the height of the image, the second dimension (60) represents the width, and the third dimension (2) corresponds to the two channels.
- **Single-Channel Convolution:** Each channel is processed independently using separate convolution kernels. Following Jiang, Kelly, and Xiu (2023)’s approach, 5×3 convolution kernels and 2×1 max pooling kernels are used due to the sparsity of images in the vertical dimension. A network block, consisting of convolutional layers, batch normalization, leaky ReLU activation, and max pooling, extracts features from each channel. An SE (squeeze-and-excitation) module is also inserted between the first and second blocks to assign higher weights to more important channels, enhancing the model’s focus on useful features. The

depth of the convolutional layers starts at 64 filters in the first block and doubles after each max pooling layer, as proposed by previous works (Simonyan, 2014). This design ensures that each channel retains its distinct information while minimizing interference.

- **Cross-Channel Convolution Fusion:** Alongside the individual channel feature extraction, the features from both channels are fused through a shared convolution kernel. This fusion step allows the model to leverage interchannel information and capture potential channel relationships, facilitating a more global feature representation. Except for including the SE module after the second block, the settings for the cross-channel convolution fusion layer mirror those of the single-channel convolution process.
- **Feature Fusion:** The features extracted from both channels are concatenated along the channel dimension, creating a comprehensive feature map that combines the independent information from each channel and the mutual information between the channels. This fusion enables the model to capture a more complete and holistic data representation.
- **Fully Connected Layer:** The concatenated features are passed through fully connected layers, which further combine and abstract the information to make predictions. To improve the model’s generalization ability and reduce overfitting, a dropout technique was applied, randomly deactivating 50% of the neurons before the fully connected layer.
- **Output Layer:** Finally, the output layer uses the softmax function to produce the probability of future price increases, which serves as the model’s final prediction.

This model introduces a dual-channel convolution and channel fusion mechanism that preserves each channel’s information independence while capturing the mutual information between channels using shared convolution kernels. This design enhances the model’s ability to extract features from multi-channel input data, allowing for more diverse and flexible feature representations.

4 Empirical analysis

This study used CNN to analyze grayscale image data representing stock market information and predict price trends for the subsequent 20 days. This chapter focuses on detailing the model training process and presenting the results.

4.1 Model training

According to the previously outlined data cleaning and processing methods, approximately 480,000 image data samples were obtained. This study used the top 30% of the data, sorted chronologically from 2000 to 2012, as the training and validation sets, while the remaining data were used as the test set. During the generation of the training and validation sets, 70% of the data was designated as the training set; the remaining 30% was used for validation.

The batch size for each training iteration was set to 128. Furthermore, to prevent overfitting from the data order during multiple training epochs, the data were randomly shuffled during training, which improved the model’s generalization ability. The label distribution of price trends (up/down) in the dataset is approximately 48:52, and this distribution is similar across both the training and validation sets, with a ratio of 49:51. Therefore, the dataset was well-balanced, and no additional rebalancing or processing of labels were necessary.

This study used the PyTorch framework to train the model, and graphics processing unit (GPU) acceleration was enabled to transfer the model to run on the GPU. Since the task in this study is a classification problem, the cross-entropy loss function was chosen, which is mathematically expressed as follows:

$$L(y, \hat{y}) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}). \quad (13)$$

Here, y is the true label, and $\hat{y}$ is the predicted label output by the model. The Adam optimizer was employed to enhance the training efficiency. Adam is known for its ability to adjust the learning rate during training, which accelerates convergence and helps to prevent overfitting. The initial learning rate was set to 1e-5. Additionally, an early stopping strategy was implemented to prevent unnecessary training once the model had reached a plateau. The training process is terminated early if the validation loss does not improve for five consecutive epochs.

Due to multiple sources of randomness in the CNN training process, specific differences in the model’s performance may be obtained from each training session. These sources of randomness include weight initialization, data shuffling, and dropout, all of which can affect the final model’s performance. Therefore, this study conducted 30 independent training sessions on the model to obtain more robust and representative experimental results. No modifications were made between

training sessions except for the random seed, which was varied to ensure that a particular random initialization did not bias the results. This approach allows a more reliable assessment of the model’s generalizability and stability across different training conditions.

4.2 Performance results of the model

4.2.1 Performance of classification

Softmax outputs predicted probabilities; hence, it is necessary to determine the classification threshold before generating the final prediction. This study chooses the minimum distance method to determine the optimal threshold. The key idea behind this method is to select the point on the ROC curve closest to the ideal point (0,1), which represents the perfect classification performance. The calculation methods for the true positive rate (TPR) and false positive rate (FPR) are given by the following formulas:

$$TPR = \frac{TP}{TP + FN} \quad (14)$$

and

$$FPR = \frac{FP}{FP + TN}. \quad (15)$$

Where:

- TP (True Positive) is the number of cases where the true situation and the predicted result are positive.
- FN (False Negative) is the number of cases where the true situation is positive, but the predicted result is negative.
- FP (False Positive) is the number of cases where the true situation is negative, but the predicted result is positive.
- TN (True Negative) is the number of cases where the true situation and the predicted result are negative.

The threshold determined by using the minimum distance method is calculated as follows:

$$threshold = \sqrt{(1 - TPR)^2 + FPR^2}. \quad (16)$$

This threshold is then applied to the model’s predicted probabilities to classify the data as positive or negative based on which side of the threshold the prediction falls. We determined the corresponding threshold in each repeated training; after 30 independent repeated training sessions, the average accuracy of the model’s judgment is 52.39%. The mean Spearman correlation coefficient between the predicted logits and the future 20-day return is 2.62%.

In addition to the classification accuracy performance, we designed a dynamic position adjustment strategy based on the model’s classification results, selecting stocks predicted to rise every 20 days for buying operations. The model uses 20 days of historical data as input images to predict the performance of the next 20 days; therefore, the investment portfolio is readjusted every 20 days. We calculated that the cumulative returns on the test set reached 240.50%.

We employ the bootstrap method to assess the statistical significance of the model’s performance. Bootstrap is a nonparametric statistical technique based on resampling that constructs a sample distribution by repeatedly sampling with replacement from the original dataset. This method allows for estimating confidence intervals or testing the statistical significance of model parameters without making assumptions about data distributions. Specifically, we performed multiple resampling to determine the mean distribution of cumulative logarithmic returns obtained from 30 repeated experiments. This approach allows us to test whether the returns from the model-selected portfolio are significantly greater than the benchmark returns (i.e., buying all stocks in the market). After performing the bootstrap test, the results show that the cumulative return of the stocks selected by the model was significantly higher than the benchmark return ($p < 0.001$). This finding indicates that the model’s stock selection strategy has substantial predictive power and outperforms a passive strategy of buying all stocks.

4.2.2 Portfolio performance

Portfolio analysis can provide a more in-depth understanding of the performance of CNN models in predicting stock movements. In this study, stocks are sorted based on the predicted “up”

probability, and the returns of different quantile groups are used to evaluate the model’s performance. The model updates its predictions every 20 days; thus, stocks are ranked based on the predicted probability values output by the model. These stocks are then divided into 10 equal groups (deciles) according to the quantiles. The average returns and volatility are then calculated for each group.

The calculation method for the average returns of each decile involves computing the average return for each decile group in the test set over time. Similarly, the average volatility for each group was calculated using the same method. Figure 4 shows that overall, the volatility of stocks remains stable between 36% and 38%. At the same time, the return shows an upward trend as the group ranking moves from low to high.

[Insert Figure 4 about here]

The lowest return rate was observed in Group 1 (2.8%), and the rate gradually increased across groups, with the maximum value appearing in Group 10 at 12.0%. This outcome suggests that the model’s ranking can distinguish the potential returns of different groups to a certain extent, with good monotonicity. The difference in returns between the lower groups (1 to 5 groups) was relatively larger than that between the higher groups (5 to 10 groups), indicating greater differences within the lower groups and a decrease in discrimination between the higher groups. Furthermore, this study specifically focuses on the performance difference between the highest- and lowest-ranking groups, namely, the long-short spread portfolio (H–L). This analysis highlights the spread between the highest and lowest return deciles, providing insights into the model’s ability to differentiate between stocks with significantly different performance potential. This study calculates the Sharpe ratio for each decile and H–L group to evaluate the risk-adjusted returns. The Sharpe ratio helps assess the return relative to the risk taken, allowing for a more comprehensive comparison of the model’s stock selection performance across different quantiles. Table 1 showed the model’s portfolio performance in terms of decile grouping under equal and weighted conditions. The table focuses on annualized average returns and Sharpe ratios for 20-day holding-period returns. The long-short H–L strategies earn excess return rate under equal weighting is 9.2% annually, which is statistically significant at the 1% level, with a Sharpe ratio (SR) of 1.026. This outcome indicates that the model effectively distinguishes between high and

low-potential stocks when using an equal weighting strategy, with the high group significantly outperforming the low group in terms of returns. Regarding market capitalization weighting, the long-short H-L strategies earn excess return is 8.2% annually, which is also economically and statistically significant with an SR of 0.600.

[Insert Table 1 about here]

5 Comparison with other input images in terms of performance

Existing research typically uses single-channel images as the input data; however, this paper explores the potential benefits of incorporating additional information by increasing the number of image channels. Specifically, we compare the performance of models trained with different image configurations, such as

1. Single-channel images without 20-day return information (i.e., using only basic stock data without return-based features).
2. Single-channel images with 20-day return information (i.e., adding return information for the past 20 days to the single-channel images).
3. Dual-channel images without 20-day return information (i.e., using two channels without return-based features).
4. One-dimensional CNN models.

This section aims to investigate the impact of channel quantity and the richness of information on model performance. By conducting experiments, we examine the following:

1. Whether dual-channel images outperform single-channel images regarding prediction accuracy and overall model performance.
2. Whether including additional information, such as 20-day return data, improves the model's ability to predict stock price movements.

3. Whether the performance of 1D-CNN models surpasses that of 2D-CNN models.

This study compares these input configurations to determine the significance of adding more channels and information to enhance the model’s prediction capabilities. The results will provide insights into how richer input data affects performance and whether improved predictions justify the increased complexity.

5.1 Comparison of single channel performance with and without 20-day return information

The input images used in the study contain the following financial information for each stock: Daily opening price, highest price, lowest price, closing price, 20-day moving average, and trading volume. These features are depicted in the images following the drawing method used by Jiang, Kelly, and Xiu (2023), which provides a standardized approach for visualizing and processing stock data—the model architecture and training methods aligned with their approach, ensuring consistency in comparing results.

The study compares the performance of the H–L group using two different input configurations:

1. Single-channel images with the aforementioned stock data.
2. Dual-channel images include the same stock data and additional 20-day return information.

To evaluate the effectiveness of each input configuration, 30 independent experiments were conducted with each configuration, allowing for a robust comparison of the model’s performance.

After completing the 30 repeated experiments, the results are summarized in Table 2. These results compare the performance of the two input configurations in terms of their ability to predict stock price movements, focusing on the performance of the H–L group. The table highlights significant differences in the return rates and SR between the two approaches.

[Insert Table 2 about here]

After bootstrap testing, the performance of the dual-channel image with 20-day returns and SR was significantly better than that of the single-channel image without 20-day returns at the 0.05 and 0.01 levels, respectively. The experimental results show that under equal weighting

conditions, the dual channel structure with 20-day returns increases return by 0.4% and SR by 0.143. Under market value weighting, the dual-channel image with 20-day return information showed significantly better returns and SR (0.01) than the single-channel image without 20-day return information. The returns and SR increased by 3.6% and 0.289, respectively.

This result has great practical significance, as investment decisions in the market often consider market value weighting factors. High market value stocks can better reflect the overall trend of market changes; therefore, the dual channel with 20-day return information performs well, indicating that additional return information helps the model more accurately capture market trends and risk-adjusted returns.

Nonetheless, the above comparison alone is still insufficient to determine whether adding new information or the dual-channel structure played a key role. Therefore, to further verify whether the amount of information or the number of channels has a greater impact on the model’s results, the next step is to compare the single-channel structure with added information and the dual-channel structure without added information. All information is drawn in the same channel, following the Jiang, Kelly, and Xiu (2023) overall method. The added 20-day return information is scaled and plotted in the top four-fifths of the graph. Table 3 summarizes the results of 30 repeated experiments.

[Insert Table 3 about here]

Bootstrap testing revealed that under equal weighting conditions, the return of a single channel with a 20-day return was higher than that of a single channel without a 20-day return; however, the increase was insignificant. At the same time, the improvement in SR was significant at the 0.01 level. Under market value weighting, the return and SR of the single channel with a 20-day return were significantly better than the single channel without a 20-day return ($p = 0.018$, $p = 0.016$). Furthermore, comparing the dual-channel image with the 20-day return information and the single-channel image with the 20-day return information, under equal weighting, the return significantly increased at the 0.1 level, and the SR significantly increased at the 0.01 level.

This result suggests that increasing the information of the input data (i.e., 20-day return information) can enhance the model’s return performance and SR under equal weight and weighted conditions, especially in the more practical market value-weighted, where this effect is particularly

significant. Conversely, the dual-channel structure shows advantages under the same information conditions, indicating that the complexity of the model structure (i.e., multi-channel features) helps the model more comprehensively capture potential relationships and dynamic features in the data.

5.2 Comparison of dual channel performance without 20-day return information

We also compared the performance of the dual channels without the 20-day return information. As introduced earlier, we employed the Ward clustering method to cluster the daily opening price, highest price, lowest price, closing price, and 20-day moving average into one category. Trading volume was included as a separate category. These two categories were plotted separately in each channel, and Table 4 summarizes the results.

[Insert Table 4 about here]

The bootstrap test results showed that increasing channels alone does not significantly improve return and SR under equal-weighted conditions. Under market value-weighted conditions, returns and SR improvements are significantly at the 0.01 level. Finally, the dual channel with 20-day return information performed better regarding return and SR than the dual-channel without information. Specifically, under equal weighted, the improvement in SR was significant at the 0.01 level, and under market value-weighted, return and SR were significant at the 0.05 and 0.1 levels, respectively. This discovery proves from another side that the increase in information and the number of channels positively affect performance. These findings provide new insights into the design of multi-factor investment and financial forecasting models, suggesting that appropriate feature information and channel design can help the model demonstrate greater adaptability in complex market environments.

5.3 Comparison with 1D-CNN performance

Finally, we also compared the one-dimensional CNN (1D-CNN) models, which can effectively capture local dependency features in time series data through local perception and weight-sharing

mechanisms. Unlike the traditional 2D-CNN for processing image data, the convolution kernel of the 1D-CNN slides along a single time axis for feature extraction. Its mathematical expression is:

$$y_t = \sigma \left(\sum_{i=1}^k w_i \cdot x_{t+i} + b \right). \quad (17)$$

Here, w_i is the weight of the convolution kernel, b is the bias term, σ is the nonlinear activation function, and k is the size of the convolution kernel. This article's input data matrix dimension is 7 times 20 (7 variables $\times$ 20 time points). The 1D-CNN model includes three convolutional modules, each with a convolutional layer, a batch normalization layer, a LeakyReLU activation function, and a max pooling layer. The first convolutional layer has 7 input channels and 64 output channels. The output channels of the following two layers are twice that of the previous layer. Finally, the fully connected layer is used for classification, and the output dimension is 2.

This article uses three methods for data preprocessing: maximum minimum normalization, volatility scaling, and Z-score normalization. For maximum minimum normalization, the opening price, highest price, lowest price, closing price, and 20-day moving average are normalized simultaneously; the trading volume and 20-day return are normalized separately. Specifically, in the 1D-CNN, the NaN data points must be assigned a value of 0, while the CNN model used earlier does not need to, as the previous model is not sensitive to NaN. Table 5 summarizes the Spearman correlation coefficients and model judgment accuracy between the predicted logit obtained by different preprocessing methods and future 20-day returns.

[Insert Table 5 about here]

Tables 6 to 8 present the experimental results. First, we compare the results using different data processing methods. The data processed by maximum minimum normalization showed good monotonicity. This outcome indicates that the model can effectively capture factor monotonicity, and the H-L group returns under equal weight are significant at the 0.1 level. After volatility scaling, the H-L returns of all weighted methods are negative, which may be due to the excessive compression of time series volatility information by volatility scaling, which weakens the model's sensitivity to extreme value signals. The Z-score normalization results are similar to volatility scaling, with negative H-L SR for both. Focusing on the weighting method reveals a sharp reversal in volatility scaling and Z-score normalization, indicating its high sensitivity to the data

scale. This nonlinear response may be due to the amplification of the high-frequency noise by the volatility weighting.

[Insert Tables 6, 7, and 8 about here]

Table 9 compares the H–L performance of the 1D-CNN with that of the 2D-CNN, showing that under different weighting conditions, the 2D-CNN has better returns and Sharpe ratios. Under equal weighting conditions, the return rate increased by 96%, and SR increased by 128%. This outcome indicates that the 2D-CNN captures richer spatial features, while the 1D-CNN can only capture information from a single dimension. Meanwhile, the convolution kernel of the 2D-CNN is two-dimensional, which means it can extract features in both directions simultaneously, capturing more complex spatial dependencies and local patterns. The 2D-CNN can learn more hierarchical spatial features when processing image data, thereby improving the model’s performance.

[Insert Table 9 about here]

6 Sensitivity experiments

This section conducts a sensitivity analysis and performs experiments on various model parameters. These parameters include changes to the number of convolution kernels, convolution layers, dropout rate, the use of BatchNorm (BN), activation function, max pooling, convolution kernel size, dilation, and stride, and the inclusion of the SE module. Table 10 summarizes the results. The table includes cross-entropy loss on both the validation set (V) and test set (T), classification accuracy, Spearman correlation coefficient on the test set, and the SR of the H–L group under equal weight and market capitalization weighting conditions. The first row of the table displays the relevant results of the benchmark model, with the parameters of the benchmark model described in parentheses in the row headings. The following column describes the parameters modified in the sensitivity experiment.

[Insert Table 10 about here]

Reducing the number of convolution kernels to 32 reduces the accuracy(Acc) and SR on the test set, indicating a decrease in the model’s ability to mine information. Increasing the number of

layers to 3 improves the performance of the SR; however, too many layers may lead to overfitting and reduce the model’s effectiveness. The change in dropout has a small impact on Acc and the correlation coefficient. When dropout is set to 0, the SR results deteriorate, while the performance is better at 0.25 and 0.50. When dropout is set to 0.75, the performance is better under the market value-weighted, which may be due to the high dropout causing the model to train on fewer features, thereby affecting the effectiveness. Using BN and leaky ReLU will result in a better SR. Modifying the pooling size to (2, 2) significantly negatively impacts performance under both equal and market value-weighted. Different combinations of expansion and stride have a certain impact on the performance and feature extractions of the model, and more intensive extraction may have better results. Adjusting the filter size appropriately can optimize the stability and accuracy of the model. The SE module slightly improved the SR under equal and market value-weighted.

7 Monte Carlo simulations

Due to the high noise ratio, revenue prediction is a challenging problem. Therefore, this study demonstrates the recognition and prediction results of the CNN models in environments with different signal-to-noise ratios through Monte Carlo simulations. To generate simulated images, this article simulates price trajectories at a frequency of five minutes to obtain daily opening, closing, highest, and lowest prices, as well as 20-day returns. In the technical analysis, the head and shoulders top (HNST) predicts future price declines, while the head and shoulders bottom (HNSB) predicts future price increases. We simulate multi-channel grayscale images of these two modes.

The functional forms of HNST and HNSB are defined as follows:

$$f_{HNST} = \begin{cases} x, 0 \leq x < 1/8 \\ -x + 1/4, 1/8 \leq x < 1/4 \\ x - 1/4, 1/4 \leq x < 1/2 \\ -x + 3/4, 1/2 \leq x < 3/4 \\ x - 3/4, 3/4 \leq x < 7/8 \\ -x + 1, 7/8 \leq x < 1 \end{cases} \quad (18)$$

and

$$f_{HNSB} = -f_{HNST}. \quad (19)$$

These modes are shown in Figure 5.

[Insert Figure 5 about here]

Based on the functions of HNST and HNSB, we use the following formula to generate the simulated prices:

$$d \ln(S_t) = \kappa (\text{mean level}_t - \ln(S_{t-1})) dt + \sigma \sqrt{dt} \varepsilon. \quad (20)$$

Here, $\kappa = 50/T$, is the mean response intensity. When generating an image with a length of 20 days, $T=20/252$, $\text{mean level}_t = 3\sigma\sqrt{T}f_{HNST/HNSB}(t/T)$. It is the mean level calculated based on the HNST or HNSB pattern function, where σ is the volatility, following $\mathcal{U}(0.2, 0.6)$, and σ is fixed in the same simulated image. Finally, ε is the noise that follows a standard normal distribution.

The simulation price without a pattern is described using the geometric Brownian motion (GBM) model:

$$d \ln(S_t) = \sigma \sqrt{dt} \varepsilon. \quad (21)$$

Here, σ is the volatility, which still follows $\mathcal{U}(0.2, 0.6)$, and ε is the noise that follows the standard normal distribution.

After obtaining the price trend chart, we extract the 20-day yield of the opening, highest, lowest, and closing prices. Specifically, to obtain a 20-day yield, it is necessary to generate data for at least 20 days before the start of the pattern. This article uses pattern-free data to fill in the data. Figure 6 shows the simulated multi-channel grayscale images.

[Insert Figure 6 about here]

7.1 Including both HNST and HNSB

This article specifies the noise ratios of 0.990, 0.992, 0.994, 0.996, 0.998, and 0.999 in the experiment, which means that only 1%, 0.8%, 0.6%, 0.4%, 0.2%, and 0.1% of the images, respectively, in the experiment contain patterns, with each pattern accounting for half of them. For different noise ratios, this article conducted 10 independent experiments. In each experiment, a total of 100000 images were generated. According to the technical analysis, the label for the HNSB image is designated as rising, and the label for the HNST image is designated as falling. The non-patterned image is randomly assigned rising and falling labels with a probability of 50% each; therefore, the rising and falling labels account for 50% of the total. By adjusting different noise ratios, the experiment aims to simulate the impact of noise on the learning performance of the model in reality. A lower noise ratio indicates a greater proportion of effective pattern information, improved image quality, and easier pattern recognition and learning by the model. However, a higher noise ratio means that most images are noisy, and the model needs to extract valuable information from this noise, increasing the difficulty of training the model.

The simulation experiment uses the same CNN model architecture design as in the empirical research. Our simulation settings are more programmatic; therefore, we reduced the number of channels from 64 and 128 to 8 and 16. The dataset was divided into training, validation, and testing sets in a 6:2:2 ratio in all experiments. We calculate the average accuracy of classification for different noise ratios with and without modes, as shown in Figure 7. The results showed that under all the noise ratios, the classification accuracy of the pattern-free images was about 50%. When the noise ratio was 0.990, the pattern-containing images only accounted for 1% of the total; however, the recognition accuracy reached 94.70%. When the noise ratio was 0.999, the recognition accuracy could reach 58.06%. The accuracy of the HNSB and HNST modes is shown

in Figure 8. When the noise ratio is 0.992~0.998, the HNST judgment accuracy is slightly higher than that of the HNSB.

Overall, CNN models have good recognition ability on images with patterns but can remain neutral on images without patterns (i.e., white noise images).

[Insert Figures 7 and 8 about here]

7.2 Only includes HNSB

This article also conducted Monte Carlo simulation experiments with only one mode, with noise ratios of 0.990, 0.992, 0.994, 0.996, 0.998, and 0.999. The images containing the modes in the experiments were all head, shoulder, and bottom modes; the other parameters were the same as described in the previous text. When the noise ratio is 0.990, 0.992, and 0.994, the classification accuracy of the CNN for the head shoulder sole mode reaches 97.80%, 96.85%, and 97.41%, respectively. However, when the noise ratio is greater than 0.998, the classification accuracy of the CNN for the head-shoulder sole mode rapidly decreases. Figure 9 shows that when the noise ratio is 0.999, the accuracy is only 53.53%.

[Insert Figure 9 about here]

7.3 Only includes HNST

The situation for experiments that only include the HNST and noise modes is similar to those that only include the HNSB. When the noise ratio is 0.990 or 0.992, CNN's classification accuracy for HNST is very high. When the noise ratio is 0.992, it can even reach 99.70%. Then, as the noise ratio increases, the accuracy rapidly decreases. Figure 10 shows that when the noise ratio was 0.999, the accuracy decreased to 53.94%.

[Insert Figure 10 about here]

8 Conclusion

This study first applied Jiang, Kelly, and Xiu (2023)’s method to the Chinese market. We then improved their method, verifying the positive impact of adding more information and channels on model performance in the Chinese market. Notably, the performance significantly improved under weighted scenarios, indicating that increasing information and channels help capture market characteristics and optimize prediction results. Moreover, our results showed the beneficial effects of adding information and increasing the number of channels on the model’s performance, offering valuable insights for future improvements to this method. Nonetheless, this study’s practical significance must be enhanced. While the model performs well in theoretical testing, further optimization is required in practical applications to increase its applicability to real market conditions.

Future research can validate the robustness of this method across different market conditions and periods, including more market factors to enhance the model’s sensitivity to market changes. Furthermore, other channel combinations or information fusion methods can be explored to achieve better performance. Additional studies can investigate weighting methods for different channels to improve model performance under various asset allocation strategies. Furthermore, applying this method to other emerging markets or specific industry data could help further assess its generalization.

Appendix A. Performance on small cap stocks

To avoid the impact of shell effect, the stocks with the lowest market value of 30% at the beginning of each image were excluded in the previous text, resulting in a reduction of approximately 180000 multi-channel grayscale image data. This screening step reduces the noise impact caused by large fluctuations in the prices of small cap stocks, but also ignores the opportunities that may be contained within them. Therefore, this section will analyze a portion of small cap stocks.

Firstly, the model was retrained for small cap stocks with the same dataset partitioning and model settings. The classification accuracy was 52.57%, and the specific performance of the decile and H-L groups is shown in Table A.1. The performance of the model on small cap stocks is still outstanding, with clear upward trends in returns and SR under equal weight and market value weighting. In particular, the SR of the H-L group weighted by market value reached 1.055, which is significantly better than the results obtained in the previous section. This indicates that small cap stocks contain a lot of opportunities. For investors, although small cap stocks have a certain liquidity risk, they still have significant potential for returns under effective modeling and timing, and are worth serving as an important supplementary direction for investment strategies.

[Insert Table A.1 about here]

In addition, this article also retrained the model using all stocks, with the same dataset partitioning and model settings. The classification accuracy was 52.66%, and the performance of the obtained decile and H-L groups is shown in Table A.2. The performance of the model on the H-L group slightly decreased, but the monotonicity was poor, and the predictive ability and stability were inferior to those after screening for small market values.

[Insert Table A.2 about here]

References

- Borges, M. A., Stepnowsky, M. A., Holt, L. H., 1977. Recall and recognition of words and pictures by adults and children. *Bulletin of the Psychonomic Society* 9, 113–114.
- Chen, X., Wu, J., Xu, M., 2021. Individual default risk measurement and prevention of China’s credit bonds — based on LSTM deep learning model. *Fudan Journal (Social Sciences)* 63, 159–173.
- Ding, X., Zhang, Y., Liu, T., Duan, J., 2015. Deep learning for event-driven stock prediction. In: *Twenty-fourth international joint conference on artificial intelligence*.
- Fama, E. F., French, K. R., 1993. Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics* 33, 3–56.
- Fama, E. F., French, K. R., 2015. A five-factor asset pricing model. *Journal of Financial Economics* 116, 1–22.
- Fischer, T., Krauss, C., 2018. Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research* 270, 654–669.
- Graham, B., 1934. *Security Analysis*. McGraw-Hill.
- Grossman, S. J., Stiglitz, J. E., 1980. On the impossibility of informationally efficient markets. *American Economic Review* 70, 393–408.
- Gu, S., Kelly, B., Xiu, D., 2020. Empirical asset pricing via machine learning. *Review of Financial Studies* 33, 2223–2273.
- Gudelek, M. U., Boluk, S. A., Ozbayoglu, A. M., 2017. A deep learning based stock trading model with 2-d cnn trend detection. In: *2017 IEEE symposium series on computational intelligence (SSCI)*, IEEE, pp. 1–8.
- Hu, G., Hu, Y., Yang, K., Yu, Z., Sung, F., Zhang, Z., Xie, F., Liu, J., Robertson, N., Hospedales, T., other, 2018a. Deep stock representation learning: From candlestick charts to investment decisions. In: *2018 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, IEEE, pp. 2706–2710.
- Hu, J., Shen, L., Sun, G., 2018b. Squeeze-and-excitation networks. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 7132–7141.
- Jensen, T. I., Kelly, B., Pedersen, L. H., 2023. Is there a replication crisis in finance? *Journal of Finance* 78, 2465–2518.
- Jiang, J., Kelly, B., Xiu, D., 2023. (Re-) imag (in) ing price trends. *Journal of Finance* 78, 3193–3249.
- Jiang, W., 2021. Applications of deep learning in stock market prediction: Recent progress. *Expert Systems with Applications* 184, 115537.
- Karpoff, J. M., 1987. The relation between price changes and trading volume: A survey. *Journal of Financial and Quantitative Analysis* 22, 109–126.
- Li, B., Shao, X.-y., Li, Y., 2019. Research on machine learning driven quantamental investing. *China Industrial Economics* 8, 61–79.

- Li, Y., Hao, Z., Lei, H., 2016. Survey of convolutional neural network. *Journal of Computer Applications* 36, 2508–2515+2565.
- Liu, J., Stambaugh, R. F., Yuan, Y., 2019. Size and value in China. *Journal of Financial Economics* 134, 48–69.
- Malkiel, B. G., 2003. The efficient market hypothesis and its critics. *Journal of Economic Perspectives* 17, 59–82.
- Markowitz, H., 1952. The utility of wealth. *Journal of Political Economy* 60, 151–158.
- Sharpe, W. F., 1964. Capital asset prices: A theory of market equilibrium under conditions of risk. *Journal of Finance* 19, 425–442.
- Simonyan, K., 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* .
- Tsay, R. S., 2005. *Analysis of Financial Time Series*. John Wiley & Sons.
- Xie, H., 2019. The Improvement and Application of LSTM in Multi-factor Quantitative Investment Model. Ph.D. thesis, Central University of Finance and Economics.
- Zhang, G. P., 2003. Time series forecasting using a hybrid ARIMA and neural network model. *Neuro-computing* 50, 159–175.
- Zhao, H., Xue, L., 2021. Research on stock forecasting based on LSTM-CNN-CBAM model. *Computer Engineering and Applications* 57, 203–207.
- Zhao, Q., Xu, W., Ji, Y., Liu, G., Zhang, W., 2020. Application of machine learning to financial asset price forecasting and allocation: A literature review. *Chinese Journal of Management* 17, 1716–1218.
- Zhu, Y., 2020. Stock price prediction using the RNN model. *Journal of Physics: Conference Series* 1650, 032103.

Figure 1: Ward clustering results

This figure shows the results of Ward clustering. The variables are clustered into two groups: the first includes daily opening price, highest price, lowest price, closing price, and the 20-day moving average; the second consists of trading volume and 20-day return. The ordering of channels was not considered in the image construction process.

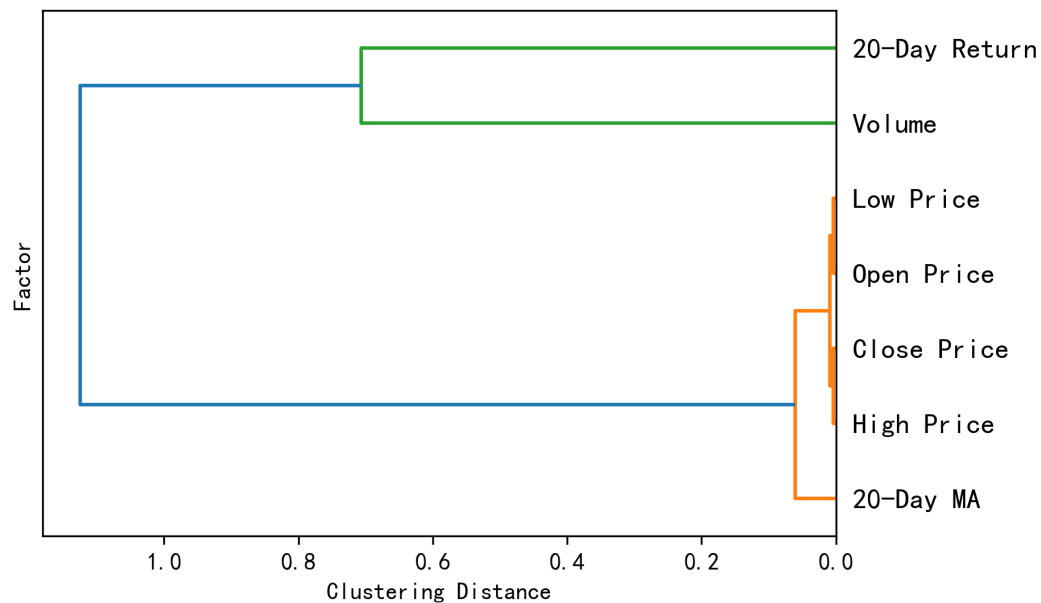


Figure 2: Multi channel grayscale image

Figures show the multi-channel grayscale image. The left part is the first channel, and the right part is the second channel. In the first channel, OHLC bars represent the daily opening, highest, lowest, and closing prices, with the 20-day moving average plotted at the middle pixels of each day, connecting these points into a line. Specifically, the data within the 20-day window is scaled such that the maximum value are positioned at the top of the image, while the minimum value is at the bottom. In the second channel, the yield curve is drawn in approximately the top four-fifths of the image, using the same method as for the 20-day moving average. The daily trading volume is represented as a bar chart in the bottom four-fifths of the image, with the maximum trading volume scaled to the top of this region. A 1-pixel black line separates the yield curve and the trading volume section to distinguish the two components. If there are missing values within the window, they are left blank in the corresponding positions.

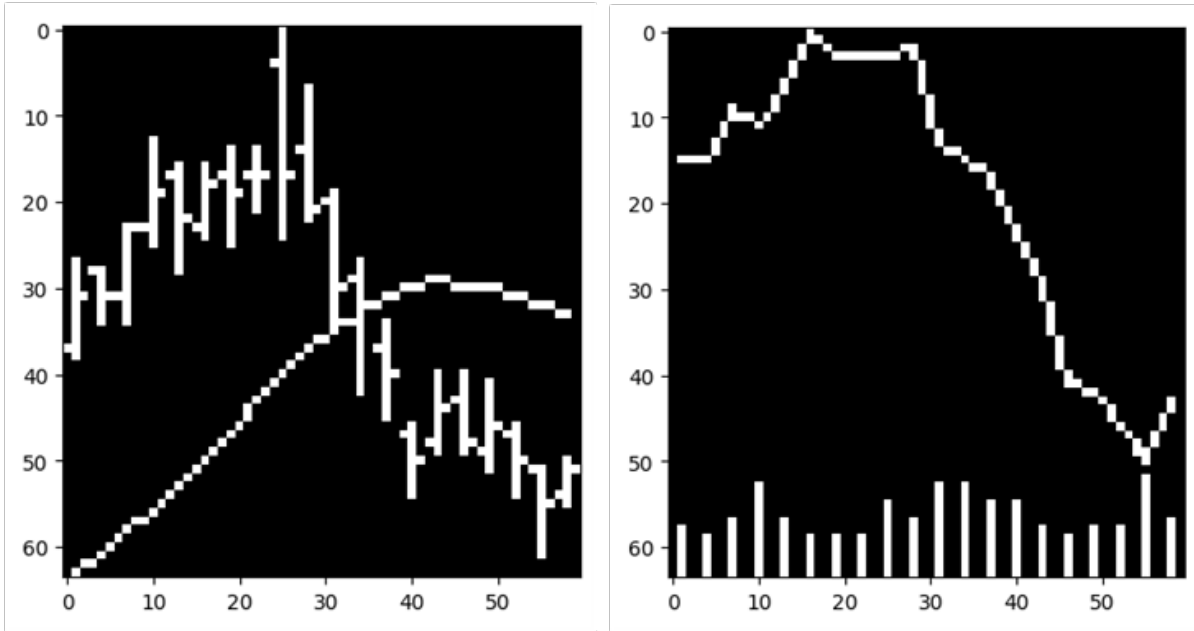


Figure 3: Model structure diagram

This figure shows a schematic diagram of the model structure. The model extracts features independently for each channel and jointly between channels, incorporating fusion operations to capture the mutual information within and between the channels. This structure enables comprehensive feature extraction through shared convolution operations while preserving the independence of each channel’s information. The model receives dual-channel input data and utilizes separate convolution kernels for each channel, performing convolution operations independently to extract features from each channel. Additionally, joint convolution operations capture mutual information between the two channels. The features extracted from both channels are then concatenated to form a comprehensive set of features, which fully connected layers process to generate the final target prediction.

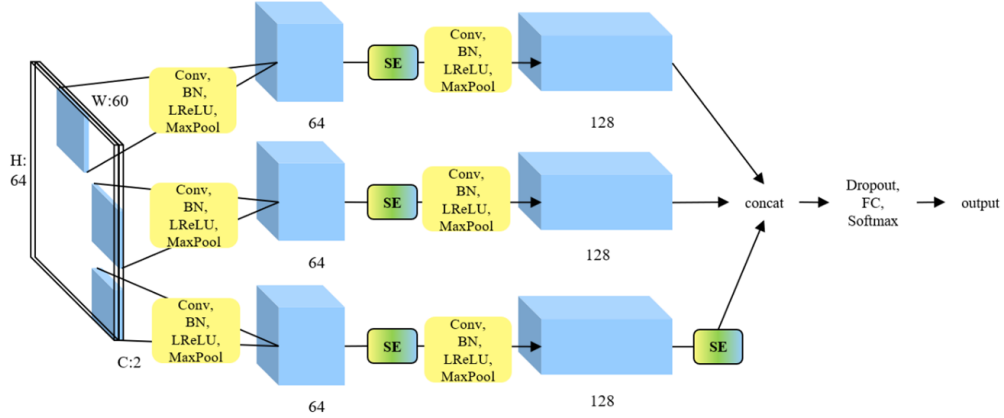


Figure 4: Average return and volatility

This figure shows the average return and volatility across groups ranked by the model. The lowest return was observed in Group 1 (2.8%), and the return increased progressively, reaching a maximum of 12.0% in Group 10. Notably, the return differences among the lower-ranked groups (Groups 1 to 5) were more pronounced than those among the higher-ranked groups (Groups 6 to 10), indicating stronger discrimination in the lower range and reduced differentiation at the higher end. In contrast, the volatility across groups remained relatively stable, ranging between 36% and 38%.

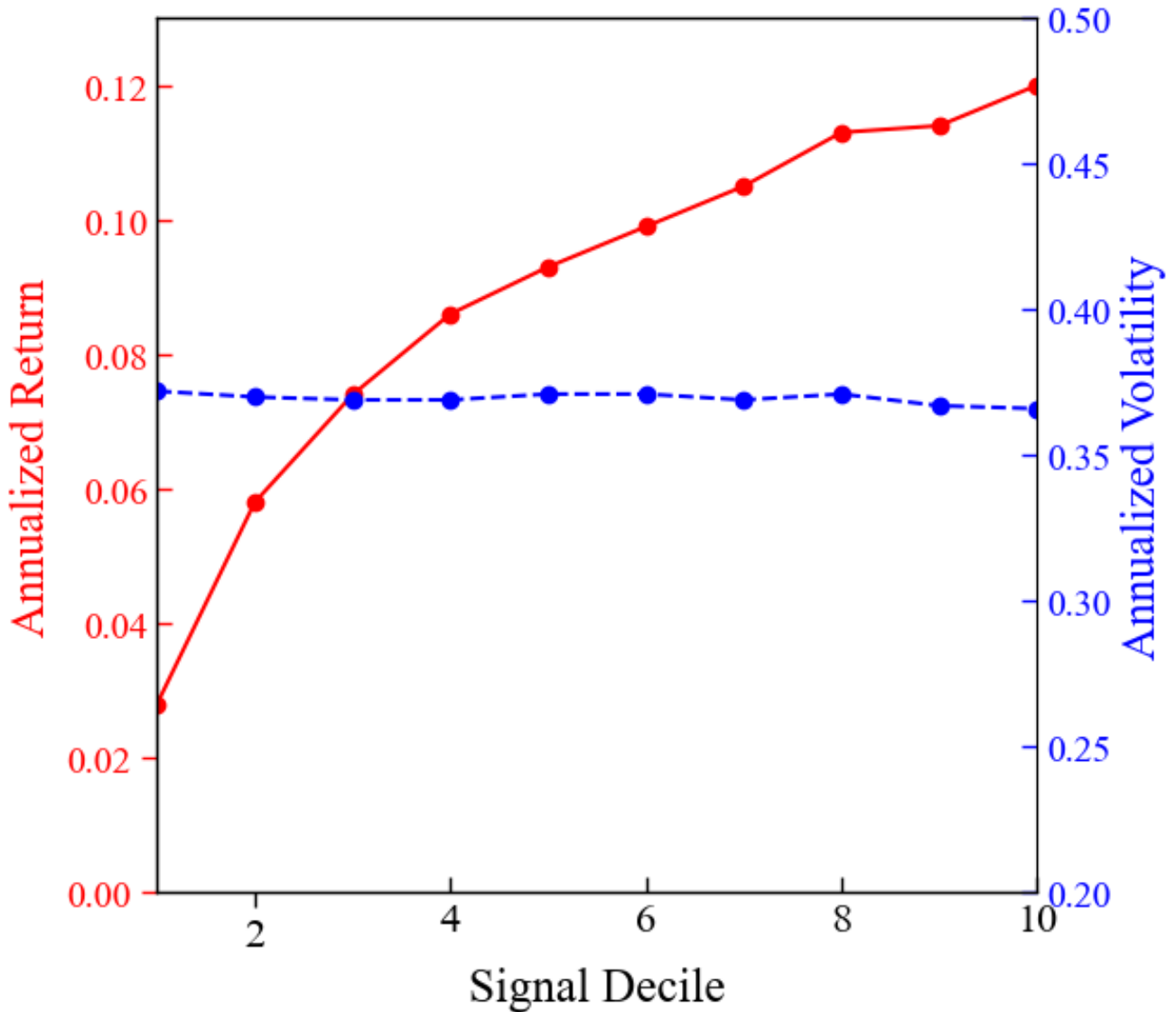


Figure 5: Functional form of HNST and HNSB

The left panel shows the NHST image, while the right panel shows the NHSB image.

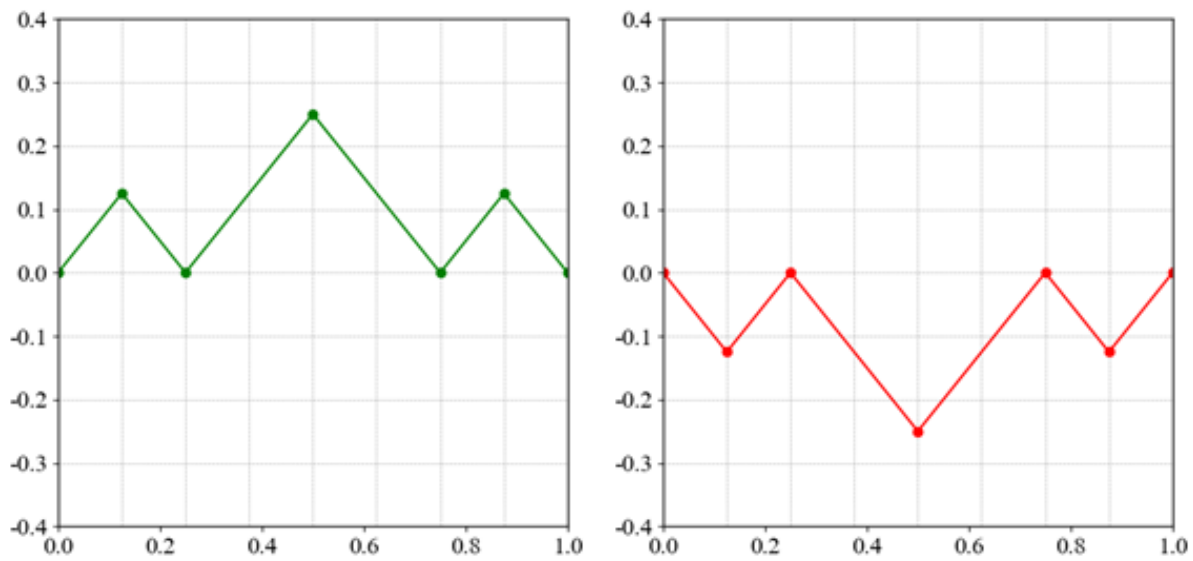


Figure 6: Comparison of different multi-channel grayscale images

Comparison of different multi-channel grayscale images. Each row corresponds to a different type: HNST, HNSB, and Pattern free (from top to bottom). The left panel in each row shows the first channel, and the right panel shows the second channel.

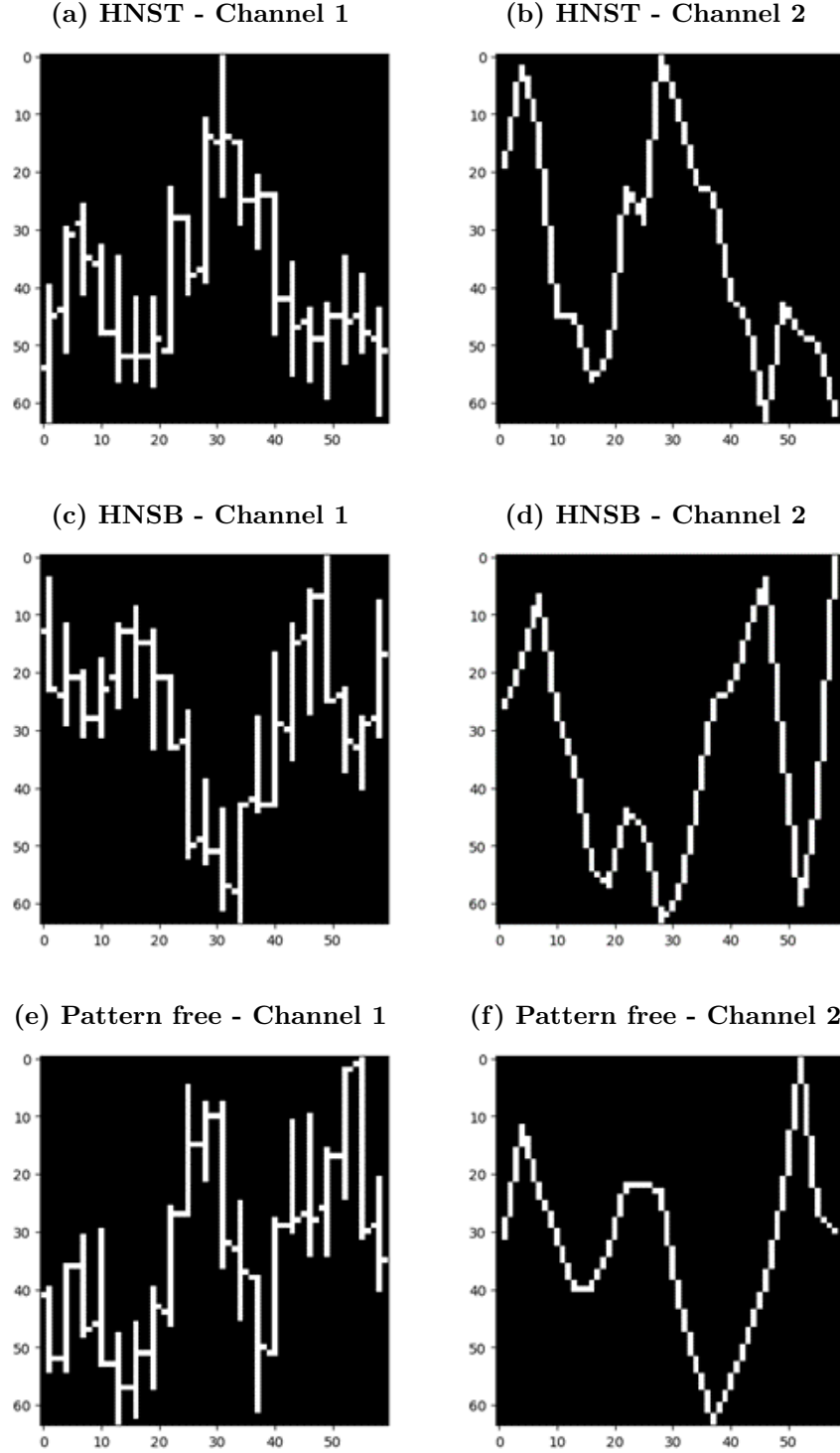


Figure 7: Prediction accuracy of HNST/HNSB and pattern-free

This figure shows the classification accuracy of the model under different noise ratios for both HNST/HNSB and pattern-free images. For HNST/HNSB images, the classification accuracy decreases as the noise ratio increases. In contrast, the accuracy for pattern-free images remains around 50% across all noise ratios.

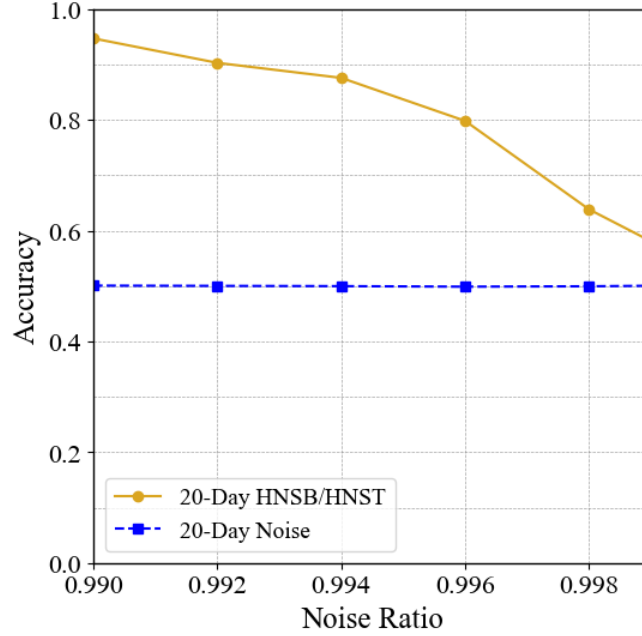


Figure 8: Prediction accuracy of HNST, HNSB, and pattern-free

This figure shows the classification accuracy of the model under different noise ratios for both HNST, HNSB and pattern-free images.

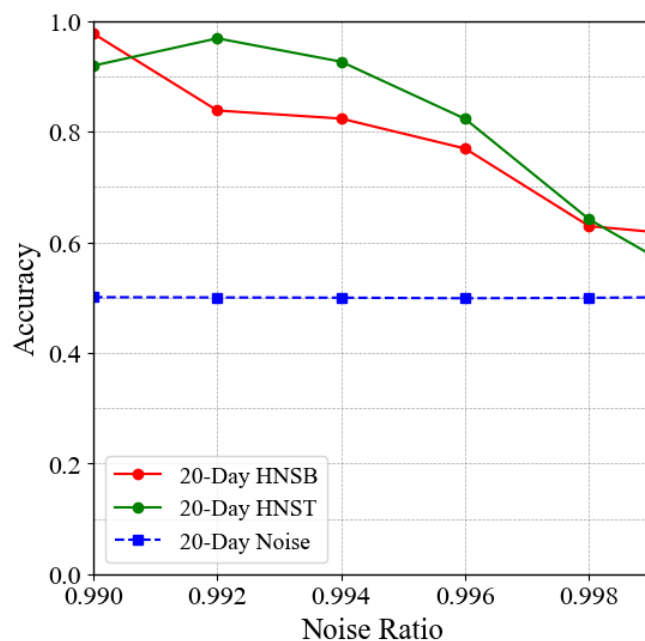


Figure 9: Prediction accuracy of HNSB and pattern-free

This figure shows the classification accuracy under different noise ratios for images containing only HNSB and pattern-free samples.

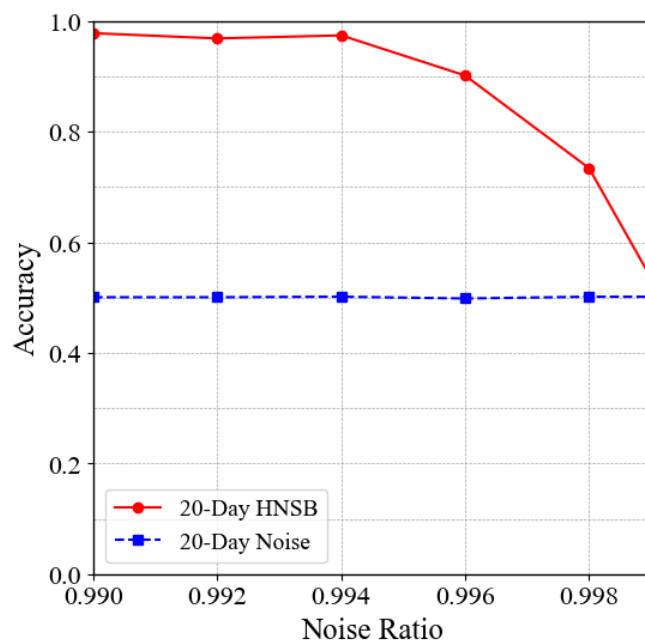


Figure 10: Prediction accuracy of HNST and pattern-free

This figure shows the classification accuracy under different noise ratios for images containing only HNST and pattern-free samples.

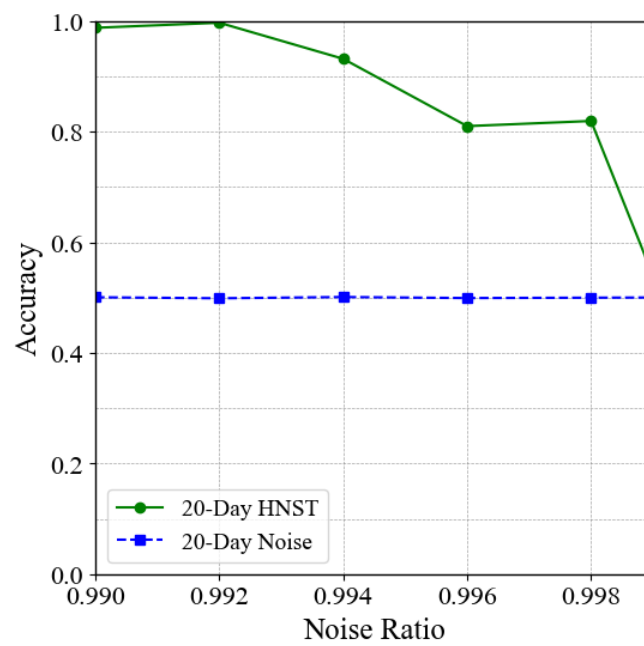


Table 1: The performance of decile portfolios

This table reports results on the performance of equal-weight and value-weight decile portfolios sorted on out-of-sample predicted “up” probability. The table reports the average holding-period return and annualized Sharpe ratio. *** and ** indicate significant at the 1% and 5% significance level, respectively.

	EW		VW	
	Ret	SR	Ret	SR
Low	0.028	0.075	0.023	0.072
2	0.058	0.157	0.042	0.133
3	0.074	0.201	0.052	0.162
4	0.086	0.232	0.068	0.213
5	0.093	0.250	0.068	0.212
6	0.099	0.267	0.065	0.206
7	0.105	0.285	0.073	0.232
8	0.113	0.305	0.080	0.253
9	0.114	0.310	0.086	0.277
High	0.120	0.329	0.105	0.340
H-L	0.092***	1.026	0.082**	0.600

Table 2: Comparison of performance to single channel images without 20-day return

This table compares the out-of-sample performance of the H–L group under both equal-weighted (EW) and value-weighted (VW) portfolios between models incorporating 20-day return with dual-channel input and those excluding 20-day return with single-channel input.

	EW		VW	
	Ret	SR	Ret	SR
Without 20-day return/single channel	0.088	0.883	0.046	0.331
With 20-day return/dual channel	0.092	1.026	0.082	0.600

Table 3: Comparison of single channel image without/with 20-day return

This table compares the out-of-sample performance of the H–L group under both equal-weighted (EW) and value-weighted (VW) portfolios among three settings: (1) excluding 20-day return with single-channel input, (2) including 20-day return with single-channel input, and (3) including 20-day return with dual-channel input.

	EW		VW	
	Ret	SR	Ret	SR
Without 20-day return/single channel	0.088	0.883	0.046	0.311
With 20-day return/single channel	0.089	0.946	0.057	0.385
With 20-day return/dual channel	0.092	1.026	0.082	0.600

Table 4: Comparison of the dual channel images without 20-day return

This table compares the out-of-sample performance of the H–L group under both equal-weighted (EW) and value-weighted (VW) portfolios among three settings: (1) excluding 20-day return with single-channel input, (2) excluding 20-day return with dual-channel input, and (3) including 20-day return with dual-channel input.

	EW		VW	
	Ret	SR	Ret	SR
Without 20-day return/single channel	0.088	0.883	0.046	0.311
Without 20-day return/dual channel	0.089	0.894	0.074	0.543
With 20-day return/dual channel	0.092	1.026	0.082	0.600

Table 5: Correlation coefficients and true rates of different preprocessing methods

This table presents the Spearman correlation coefficients and model classification accuracy between the predicted logits and future 20-day returns under the 1D-CNN framework, after applying different data preprocessing methods: maximum-minimum normalization, volatility scaling, and Z-score normalization.

	Max-Min	Vol Scaling	Z-score
Coef	0.016	-0.011	-0.000
Truerate	0.513	0.500	0.506

Table 6: The training results after max-min normalization processing

This table reports the portfolio performance of the 1D-CNN model under equal-weighted (EW) and value-weighted (VW) portfolios after applying min-max normalization. It presents the average holding-period return and the annualized Sharpe ratio under both weighting methods. Significance at the 10% level is indicated by *.

	EW		VW	
	Ret	SR	Ret	SR
Low	0.065	0.179	0.068	0.221
2	0.072	0.199	0.058	0.186
3	0.074	0.202	0.051	0.164
4	0.078	0.212	0.052	0.166
5	0.084	0.229	0.055	0.175
6	0.092	0.249	0.057	0.179
7	0.098	0.266	0.067	0.213
8	0.104	0.278	0.064	0.202
9	0.110	0.295	0.069	0.217
High	0.112	0.298	0.083	0.259
H-L	0.047*	0.450	0.015	0.093

Table 7: The training results after volatility scaling processing

This table reports the portfolio performance of the 1D-CNN model under equal-weighted (EW) and value-weighted (VW) portfolios after applying volatility scaling processing. It presents the average holding-period return and the annualized Sharpe ratio under both weighting methods.

	EW		VW	
	Ret	SR	Ret	SR
Low	0.055	0.146	0.061	0.202
2	0.097	0.260	0.087	0.258
3	0.109	0.294	0.097	0.285
4	0.118	0.319	0.097	0.286
5	0.117	0.321	0.093	0.275
6	0.113	0.312	0.097	0.289
7	0.101	0.281	0.088	0.266
8	0.089	0.246	0.077	0.233
9	0.068	0.187	0.064	0.196
High	0.023	0.061	0.053	0.187
H-L	-0.032	-0.239	-0.007	-0.052

Table 8: The training results after Z-score normalization processing

This table reports the portfolio performance of the 1D-CNN model under equal-weighted (EW) and value-weighted (VW) portfolios after applying Z-score normalization processing. It presents the average holding-period return and the annualized Sharpe ratio under both weighting methods.

	EW		VW	
	Ret	SR	Ret	SR
Low	0.032	0.080	0.052	0.173
2	0.084	0.228	0.073	0.227
3	0.095	0.269	0.074	0.233
4	0.104	0.297	0.077	0.244
5	0.108	0.311	0.084	0.268
6	0.110	0.315	0.083	0.266
7	0.111	0.314	0.085	0.270
8	0.104	0.288	0.086	0.271
9	0.099	0.263	0.084	0.261
High	0.042	0.102	0.048	0.149
H-L	0.010	-0.004	0.090	-0.035

Table 9: Comparison between 1D-CNN and 2D-CNN

This table compares the H–L group performance under equal-weighted and value-weighted portfolios between the 1D-CNN model with min-max normalization and the 2D-CNN model.

	EW		VW	
	Ret	SR	Ret	SR
1D-CNN	0.047	0.450	0.015	0.093
2D-CNN	0.092	1.026	0.082	0.600

Table 10: Results of sensitivity experiments

This table summarizes the experimental results. It reports the cross-entropy loss on the validation set (V) and test set (T), classification accuracy (Acc.), and the Spearman correlation coefficient (Corr.) between predicted logits and future returns on the test set. In addition, the table provides the Sharpe ratio (SR) of the H-L group under equal-weighted (EW) and value-weighted (VW) conditions. The experiment involves modifying architectural components such as filters, layers, dropout, batch normalization (BN), activation function, max-pooling size, dilation/stride, filter size, and squeeze-and-excitation (SE). Values in parentheses in the first column indicate baseline settings, while the second column shows the modified configuration.

		Loss		Acc.		Corr.	SR	
		V	T	V	T		EW	VW
Baseline		0.740	0.720	0.490	0.524	0.026	1.026	0.600
Filter(64)	32	0.730	0.718	0.494	0.523	0.025	0.987	0.573
Layers(2)	3	0.745	0.720	0.484	0.524	0.029	1.276	0.638
	0	0.722	0.712	0.493	0.524	0.025	0.986	0.542
Dropout(0.50)	0.25	0.733	0.719	0.488	0.524	0.024	0.937	0.604
	0.75	0.740	0.732	0.495	0.523	0.026	0.893	0.628
BN(Yes)	No	0.785	0.757	0.486	0.524	0.029	0.970	0.492
Activation(LReLU)	ReLU	0.725	0.711	0.493	0.524	0.029	0.973	0.496
Max-pool Size(2,1)	(2,2)	0.741	0.724	0.485	0.523	0.020	0.598	0.056
	(2,1)/(1,1)	0.738	0.717	0.489	0.524	0.036	1.230	0.728
Dilation/Stride(2,1)/(3,1)	(1,1)/(3,1)	0.733	0.717	0.497	0.525	0.026	0.966	0.562
	(1,1)/(1,1)	0.726	0.718	0.496	0.527	0.033	1.066	0.775
	(3×3)	0.741	0.721	0.499	0.524	0.024	0.753	0.355
FilterSize(5×3)	(7×3)	0.751	0.719	0.498	0.524	0.022	0.899	0.569
SE(Yes)	No	0.740	0.723	0.493	0.523	0.029	0.965	0.476

Table A.1: The small cap stock performance of decile portfolios

This table presents the portfolio performance of retraining the model specifically on small-cap stocks under the same data split and model configuration. It reports the return (Ret) and Sharpe ratio (SR) of the H–L group under both equal-weighted (EW) and market value-weighted (VW) conditions. Significance at the 1% level is indicated by ***.

	EW		VW	
	Ret	SR	Ret	SR
Low	0.152	0.365	0.146	0.360
2	0.185	0.445	0.180	0.438
3	0.186	0.458	0.187	0.478
4	0.211	0.511	0.202	0.501
5	0.212	0.531	0.216	0.545
6	0.218	0.551	0.213	0.543
7	0.226	0.568	0.227	0.583
8	0.243	0.630	0.236	0.624
9	0.265	0.683	0.263	0.690
High	0.257	0.664	0.257	0.677
H-L	0.106***	0.961	0.111***	1.055

Table A.2: The performance of decile portfolios including all stocks

This table presents the portfolio performance of retraining the model on all stocks under the same data split and model configuration. It reports the return (Ret) and Sharpe ratio (SR) of the H–L group under both equal-weighted (EW) and market capitalization-weighted (VW) conditions. Significance at the 1% and 5% levels is indicated by *** and **, respectively.

	EW		VW	
	Ret	SR	Ret	SR
Low	0.058	0.150	0.031	0.097
2	0.094	0.243	0.036	0.112
3	0.104	0.266	0.035	0.108
4	0.113	0.295	0.061	0.189
5	0.118	0.307	0.074	0.231
6	0.136	0.354	0.064	0.200
7	0.153	0.401	0.104	0.322
8	0.144	0.380	0.063	0.197
9	0.163	0.429	0.125	0.395
High	0.153	0.405	0.114	0.362
H-L	0.095***	0.975	0.083**	0.591